JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Class ConcurrentLinkedQueue<E>

java.lang.Object
  java.util.AbstractCollection<E>
    java.util.AbstractQueue<E>
      java.util.concurrent.ConcurrentLinkedQueue<E>

Type Parameters:

`E - the type of elements held in this collection`

All Implemented Interfaces:

`Serializable, Iterable<E>, Collection<E>, Queue<E>`

```
public class ConcurrentLinkedQueue<E>
extends AbstractQueue<E>
implements Queue<E>, Serializable
```

An unbounded thread-safe queue based on linked nodes. This queue orders elements FIFO (first-in-first-out). The *head* of the queue is that element that has been on the queue the longest time. The *tail* of the queue is that element that has been on the queue the shortest time. New elements are inserted at the tail of the queue, and the queue retrieval operations obtain elements at the head of the queue. A `ConcurrentLinkedQueue` is an appropriate choice when many threads will share access to a common collection. Like most other concurrent collection implementations, this class does not permit the use of `null` elements.

This implementation employs an efficient *non-blocking* algorithm based on one described in Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms by Maged M. Michael and Michael L. Scott.

Iterators are *weakly consistent*, returning elements reflecting the state of the queue at some point at or since the creation of the iterator. They do *not* throw `ConcurrentModificationException`, and may proceed concurrently with other operations. Elements contained in the queue since the creation of the iterator will be returned exactly once.

Beware that, unlike in most collections, the `size` method is *NOT* a constant-time operation. Because of the asynchronous nature of these queues, determining the current number of elements requires a traversal of the elements, and so may report inaccurate results if this collection is modified during traversal. Additionally, the bulk operations `addAll`, `removeAll`, `retainAll`, `containsAll`, `equals`, and `toArray` are *not* guaranteed to be performed atomically. For example, an iterator operating concurrently with an `addAll` operation might view only some of the added elements.

This class and its iterator implement all of the *optional* methods of the `Queue` and `Iterator` interfaces.

Memory consistency effects: As with other concurrent collections, actions in a thread prior to placing an object into a `ConcurrentLinkedQueue` *happen-before* actions subsequent to the access or removal of that element from the `ConcurrentLinkedQueue` in another thread.

This class is a member of the Java Collections Framework.

Since:

`1.5`

See Also:

`Serialized Form`

Constructor Summary

Constructors

Constructor	Description
`ConcurrentLinkedQueue()`	Creates a `ConcurrentLinkedQueue` that is initially empty.
`ConcurrentLinkedQueue(Collection<? extends E> c)`	Creates a `ConcurrentLinkedQueue` initially containing the elements of the given collection, added in traversal order of the collection's iterator.

Method Summary

Concrete Methods

Modifier and Type	Method	Description

boolean	**add**(**E** e)	Inserts the specified element at the tail of this queue.
boolean	**addAll**(**Collection**<? extends **E**> c)	Appends all of the elements in the specified collection to the end of this queue, in the order that they are returned by the specified collection's iterator.
boolean	**contains**(**Object** o)	Returns `true` if this queue contains the specified element.
boolean	**isEmpty**()	Returns `true` if this queue contains no elements.
Iterator<**E**>	**iterator**()	Returns an iterator over the elements in this queue in proper sequence.
boolean	**offer**(**E** e)	Inserts the specified element at the tail of this queue.
E	**peek**()	Retrieves, but does not remove, the head of this queue, or returns `null` if this queue is empty.
E	**poll**()	Retrieves and removes the head of this queue, or returns `null` if this queue is empty.
boolean	**remove**(**Object** o)	Removes a single instance of the specified element from this queue, if it is present.
int	**size**()	Returns the number of elements in this queue.
Spliterator<**E**>	**spliterator**()	Returns a **Spliterator** over the elements in this queue.
Object[]	**toArray**()	Returns an array containing all of the elements in this queue, in proper sequence.
<T> T[]	**toArray**(T[] a)	Returns an array containing all of the elements in this queue, in proper sequence; the runtime

		type of the returned array is that of the specified array.

Methods inherited from class java.util.AbstractQueue

clear, element, remove

Methods inherited from class java.util.AbstractCollection

containsAll, removeAll, retainAll, toString

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.util.Queue

element, remove

Methods inherited from interface java.util.Collection

clear, containsAll, equals, hashCode, parallelStream, removeAll, removeIf, retainAll, stream

Methods inherited from interface java.lang.Iterable

forEach

Constructor Detail

ConcurrentLinkedQueue

```
public ConcurrentLinkedQueue()
```

Creates a `ConcurrentLinkedQueue` that is initially empty.

ConcurrentLinkedQueue

```
public ConcurrentLinkedQueue(Collection<? extends E> c)
```

Creates a `ConcurrentLinkedQueue` initially containing the elements of the given collection, added in traversal order of the collection's iterator.

Parameters:

c - the collection of elements to initially contain

Throws:

NullPointerException - if the specified collection or any of its elements are null

Method Detail

add

public boolean add(E e)

Inserts the specified element at the tail of this queue. As the queue is unbounded, this method will never throw IllegalStateException or return false.

Specified by:

add in interface Collection<E>

Specified by:

add in interface Queue<E>

Overrides:

add in class AbstractQueue<E>

Parameters:

e - the element to add

Returns:

true (as specified by Collection.add(E))

Throws:

NullPointerException - if the specified element is null

offer

public boolean offer(E e)

Inserts the specified element at the tail of this queue. As the queue is unbounded, this method will never return false.

Specified by:

offer in interface Queue<E>

Parameters:

e - the element to add

Returns:

true (as specified by Queue.offer(E))

Throws:

`NullPointerException` - if the specified element is null

poll

```
public E poll()
```

Description copied from interface: Queue

Retrieves and removes the head of this queue, or returns `null` if this queue is empty.

Specified by:

`poll` in interface `Queue<E>`

Returns:

the head of this queue, or null if this queue is empty

peek

```
public E peek()
```

Description copied from interface: Queue

Retrieves, but does not remove, the head of this queue, or returns `null` if this queue is empty.

Specified by:

`peek` in interface `Queue<E>`

Returns:

the head of this queue, or null if this queue is empty

isEmpty

```
public boolean isEmpty()
```

Returns `true` if this queue contains no elements.

Specified by:

`isEmpty` in interface `Collection<E>`

Overrides:

`isEmpty` in class `AbstractCollection<E>`

Returns:

true if this queue contains no elements

size

```
public int size()
```

Returns the number of elements in this queue. If this queue contains more than `Integer.MAX_VALUE` elements, returns `Integer.MAX_VALUE`.

Beware that, unlike in most collections, this method is *NOT* a constant-time operation. Because of the asynchronous nature of these queues, determining the current number of elements requires an O(n) traversal. Additionally, if elements are added or removed during execution of this method, the returned result may be inaccurate. Thus, this method is typically not very useful in concurrent applications.

Specified by:

`size` in interface `Collection<E>`

Specified by:

`size` in class `AbstractCollection<E>`

Returns:

the number of elements in this queue

contains

```
public boolean contains(Object o)
```

Returns `true` if this queue contains the specified element. More formally, returns `true` if and only if this queue contains at least one element `e` such that `o.equals(e)`.

Specified by:

`contains` in interface `Collection<E>`

Overrides:

`contains` in class `AbstractCollection<E>`

Parameters:

`o` - object to be checked for containment in this queue

Returns:

true if this queue contains the specified element

remove

```
public boolean remove(Object o)
```

Removes a single instance of the specified element from this queue, if it is present. More formally, removes an element `e` such that `o.equals(e)`, if this

queue contains one or more such elements. Returns `true` if this queue contained the specified element (or equivalently, if this queue changed as a result of the call).

Specified by:

`remove` in interface `Collection<E>`

Overrides:

`remove` in class `AbstractCollection<E>`

Parameters:

`o` - element to be removed from this queue, if present

Returns:

`true` if this queue changed as a result of the call

addAll

```
public boolean addAll(Collection<? extends E> c)
```

Appends all of the elements in the specified collection to the end of this queue, in the order that they are returned by the specified collection's iterator. Attempts to `addAll` of a queue to itself result in `IllegalArgumentException`.

Specified by:

`addAll` in interface `Collection<E>`

Overrides:

`addAll` in class `AbstractQueue<E>`

Parameters:

`c` - the elements to be inserted into this queue

Returns:

`true` if this queue changed as a result of the call

Throws:

`NullPointerException` - if the specified collection or any of its elements are null

`IllegalArgumentException` - if the collection is this queue

See Also:

`AbstractQueue.add(Object)`

toArray

```
public Object[] toArray()
```

Returns an array containing all of the elements in this queue, in proper sequence.

The returned array will be "safe" in that no references to it are maintained by this queue. (In other words, this method must allocate a new array). The caller is thus free to modify the returned array.

This method acts as bridge between array-based and collection-based APIs.

Specified by:

`toArray` in interface `Collection<E>`

Overrides:

`toArray` in class `AbstractCollection<E>`

Returns:

an array containing all of the elements in this queue

toArray

```
public <T> T[] toArray(T[] a)
```

Returns an array containing all of the elements in this queue, in proper sequence; the runtime type of the returned array is that of the specified array. If the queue fits in the specified array, it is returned therein. Otherwise, a new array is allocated with the runtime type of the specified array and the size of this queue.

If this queue fits in the specified array with room to spare (i.e., the array has more elements than this queue), the element in the array immediately following the end of the queue is set to `null`.

Like the `toArray()` method, this method acts as bridge between array-based and collection-based APIs. Further, this method allows precise control over the runtime type of the output array, and may, under certain circumstances, be used to save allocation costs.

Suppose x is a queue known to contain only strings. The following code can be used to dump the queue into a newly allocated array of `String`:

```
  String[] y = x.toArray(new String[0]);
```

Note that `toArray(new Object[0])` is identical in function to `toArray()`.

Specified by:

`toArray` in interface `Collection<E>`

Overrides:

`toArray` in class `AbstractCollection<E>`

Type Parameters:

T - the runtime type of the array to contain the collection

Parameters:

`a - the array into which the elements of the queue are to be stored, if it is big enough; otherwise, a new array of the same runtime type is allocated for this purpose`

Returns:

`an array containing all of the elements in this queue`

Throws:

`ArrayStoreException - if the runtime type of the specified array is not a supertype of the runtime type of every element in this queue`

`NullPointerException - if the specified array is null`

iterator

```
public Iterator<E> iterator()
```

Returns an iterator over the elements in this queue in proper sequence. The elements will be returned in order from first (head) to last (tail).

The returned iterator is *weakly consistent*.

Specified by:

`iterator in interface Iterable<E>`

Specified by:

`iterator in interface Collection<E>`

Specified by:

`iterator in class AbstractCollection<E>`

Returns:

`an iterator over the elements in this queue in proper sequence`

spliterator

```
public Spliterator<E> spliterator()
```

Returns a `Spliterator` over the elements in this queue.

The returned spliterator is *weakly consistent*.

The `Spliterator` reports `Spliterator.CONCURRENT`, `Spliterator.ORDERED`, and `Spliterator.NONNULL`.

Specified by:

`spliterator in interface Iterable<E>`

Specified by:

spliterator in interface Collection<E>

Implementation Note:

The Spliterator implements trySplit to permit limited parallelism.

Returns:

a Spliterator over the elements in this queue

Since:

1.8

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.